



LANDING ON PLANET **DiSHACLed**

FOR MISSION
survey on commonly used pipelines

Sirus NV

Verlorenbroodstraat 122 Bus 8
9820 Merelbeke
info@sirus.be
+32 9 335 76 43
www.sirus.be

Thomas Carsten

Data Engineer
thomas.carsten@sirus.be



TABLE OF CONTENTS

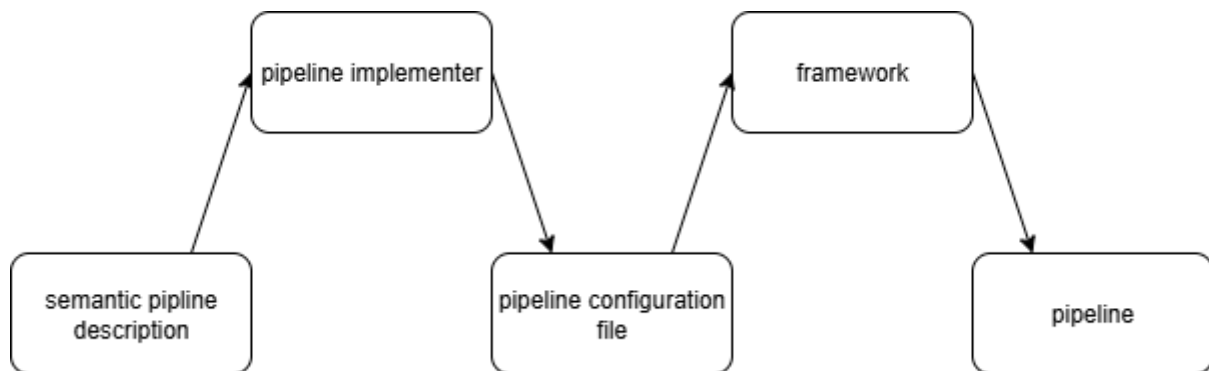
Introduction.....	3
Objective.....	4
Part 1: Assessing the state-of-the-art.....	4
1.1 Included frameworks	4
1.2 Derived Categories.....	5
1.3 Framework comparison.....	7
Part 2: Survey	8
Conclusion.....	9

INTRODUCTION

Data pipelines are automated workflows that ingest, transform, and publish data in order to provide access to information of interest at a target location. It is the experience of many data engineers today that building and maintaining these pipelines is cumbersome and time-consuming. Although a plethora of tools and frameworks exist for developing data pipelines, it often remains manual work. Oftentimes custom logic is written to clean the data, implement business decisions, and transform data into a desired format and structure.

While such manual crafting of data pipelines provides great flexibility to tailor pipelines to the needs of a particular project, it also comes with a range of disadvantages. Hard-coding custom logic in such a way is not only time-consuming, it limits reusability. This is because a certain context may be required for custom logic to execute properly. Custom logic may also not always be implemented according to best practices, thus missing the performance and robustness of enterprise-grade solutions. Furthermore, by directly embedding business decisions or other logic in code, this approach provides no clear separation between its *intent* and its *implementation*. This not only lacks transparency without further documentation, it also limits the transfer of understanding of the custom logic in case the owner responsible for the pipeline changes.

For these reasons, it is an expressed goal of the DiSHACled project to specify and demonstrate an approach to creating data pipelines, which can tackle all of these limitations at once. **The core idea is that pipelines should be composed as arrangements of modular data transformation steps, whereby each step acts on data in a well-defined, configurable way. Furthermore, these pipelines, and the transformation steps they consist of, are semantically described. This allows for the execution of the pipeline to be achieved through interpreting and implementing its semantic description. As such, a separation between *intent* and *implementation* is achieved. We call such solution the “Generic Toolchain Architecture”.**



The Generic Toolchain Architecture has a number of advantages. Semantic descriptions of transformation steps, or entire pipelines, provide a high level overview of the custom logic that is being implemented. This not only increases readability, it also makes transformation steps and pipelines *discoverable* and *shareable*. It increases efficiency in that each transformation step only needs to be developed once, as configuration should be sufficient to tailor a transformation step to a specific use case. Moreover, since the *implementation* of transformation steps are taken care of by applications that interpret the semantic pipeline description, different implementations for one and the same intended transformation are possible. This is advantageous because it also allows transformation steps in a pipeline to be taken care of by different frameworks, hence breaking silos. Finally, since pipelines are semantically described, they can take advantage of the entire Linked Data stack, such as validating pipeline definitions before their implementation or querying for suited transformation steps for a given data input. It is also possible to link datasets to pipelines that generated them, hence providing full provenance.

OBJECTIVE

It is the objective of this survey to assess the current state-of-the-art of creating data pipelines. Specifically, we want to know which transformation steps and pipeline patterns are most commonly used in the Linked Data space. This allows us to assess where the biggest needs lie for providing reusable components that can be used in data pipelines. It also allows us to assess whether our plans for developing the Generic Toolchain Architecture is able to cover commonly used pipeline patterns.

For this purpose, we conducted the survey in two parts. In Part 1, we identified frameworks capable of transforming Linked Data by chaining modular transformation steps to a workflow. We scraped and categorized names and descriptions of these transformation steps per framework. This allowed us to evaluate the functional range of transformation steps used in pipelines. In doing so, we arrived at a conceptual distinction between different categories of transformation steps that are typically used within Linked Data pipelines. We also assessed each frameworks' capability to serve as foundational building block for the Generic Toolchain Architecture.

In Part 2, we conducted an online survey which allowed participants to interactively sketch out pipelines via an intuitive drag-and-drop interface. This allowed us to empirically assess which types of transformation steps were most commonly used and which pipeline patterns were common across our survey participants. As such, Part 1 was an important forerunner to Part 2, in that we categorized participants' responses by the categories identified in Part 1.

PART 1: ASSESSING THE STATE-OF-THE-ART

1.1 INCLUDED FRAMEWORKS

To assess the current state-of-the-art, we formulated a number of requirements that frameworks need to fulfill to be included in this survey. To qualify as a potential foundational building block for the Generic Toolchain Architecture a framework must

- Consist of modular transformation steps, which can be chained in various ways
- Be capable of transforming RDF data
- Be fully open-source

We included 10 frameworks in this survey, which fulfil these requirements.

Table 1.

framework	number of scraped items
Apache Nifi	290
Knime	273
semantic.works	259
LinkedPipesETL	77
bernard59	60
RDFConnect	31
RDFpro	20
LDIO	18
Fintan	16
chimera	11
	1055

1.2 DERIVED CATEGORIES

We scraped the names, descriptions, and original categories (if available), of the 1055 transformation steps across the 10 frameworks. Based on these, we reformulated each transformation step following the format action verb – proposition – object. Examples are “write to dataset” or “validate against schema”. Creating a novel item set in this way allowed to quickly identify common themes across the items, leading us to generalize transformation steps across 5 categories, which can be further split up into 15 subcategories. An overview of the 15 categories together with example items can be found in Table 2.

Table 2

parent category	category			
In/Out	Read Data	In	description action verbs examples	Processors that load data into the pipeline for further processing. <i>capture, consume, extract, fetch, get, import, ingest, load, open, parse, pull, read, retrieve</i> open a file to load its contents into memory, receive data packets from a messaging system or queue
		Out	description action verbs examples	Processors that export data from the pipeline for storage or receipt. <i>dump, insert, materialize, package, post, publish, push, put, send, stream, write</i> transfer data from a source system to a target location, save data onto a physical storage
	File Handling		description action verbs examples	Processors that commit changes on the storage structure of the files holding the data. <i>copy, create, delete, find, list, move, purge, remove, rename</i> move file from one location to another, create a duplicate of a file
	Communication		description action verbs examples	Processors that establish a connection between two systems, allowing the transfer of data. <i>authenticate, connect, handle, listen, request, sign, verify</i> establish a connection between services for data transfer, tap into the flow of continuous data so that it is received when new data arrives
	Data Representation		description action verbs examples	Processors that change how data is represented without changing the content of the data. <i>compress, convert, encode, encrypt, serialize, unpack, unzip</i> change a file from one format to another, convert the data structure into a predefined format
Refinement	Data Selection		description action verbs examples	Processors which select a subset of the data for further processing. <i>lookup, combine, construct, filter, query, sample, scan, select</i> retain only the data entries that meet specific criteria, while excluding others, delete repeated entries from a dataset to ensure each entry is unique
	Data Manipulation		description action verbs examples	Any processor that changes the actual content of the original data in some way. <i>anonymize, append, concatenate, deduplicate, enrich, expand, flatten, hash, mark, modify, replace, set, substitute, tag, transform, translate, update</i> correct errors and remove inconsistencies in data, correct errors and remove inconsistencies in data
	Data Validation		description action verbs examples	Processors that check whether the incoming data fulfils predefined criteria. <i>ask, validate</i> check whether data conforms to a predefined structure, verify whether all required data is present in datasets

Parent category	category		
Insight	Data Summarization	description	Processors that calculate metrics for the data to describe it.
		action verbs examples	<i>calculate, compute, count, describe, score</i> find and count how often certain data entries appear within a dataset, calculate a metric that describes the data in some way
	Data Inference	description	Processors aimed at making predictions or drawing conclusions based on the data.
		action verbs examples	<i>analyze, classify, cluster, comprehend, estimate, identify, predict, test</i> perform a statistical test to evaluate whether patterns in data are due to chance, assign items to categories based on its characteristics
Pipeline Execution	Data Visualization	description	Processors which visualize the data for inspection.
		action verbs examples	<i>explore, inspect, plot, view</i> visualize data with a graphic to reveal patterns, trends and relationships, Show distributions, outliers, and missing values to assess data quality
	Data Learning	description	Processors which train a model based on the data.
		action verbs examples	<i>embed, finetune, initialize, label, layer, learn, pool</i> teach a machine learning algorithm to recognize patterns in data, retrain a model with task-specific data to improve its performance on that task
	Flow Control	description	Processors that control the execution of the pipeline.
		action verbs examples	<i>assign, batch, buffer, control, detect, distribute, end, enforce, fork, join, limit, merge, monitor, notify, partition, retry, route, segment, split, start, wait</i> track the operations and performance of a data pipeline, spread workload across systems to optimize performance
	Execute Process	description	Processors which allow the execution of a command by taking care of the implementation.
		action verbs examples	<i>apply, compile, execute, invoke, map, prompt, run</i> run a script to have commands executed on data and return its output, start a task which performs operations on data in a predefined way
	Data Generation	description action verbs examples	Processors which generate new data. <i>debug, generate</i> generate mock data for testing or debugging purposes, create rare, outlier, or edge-condition data for robustness testing

1.3 FRAMEWORK COMPARISON

As a final step of Part 1, we assessed in how far each of the 10 frameworks may be suited as potential foundational building block for developing the Generic Toolchain Architecture. With foundational building block we mean the possibility to use the framework for implementing any semantic description of a pipeline through building and executing a corresponding pipeline.

In addition to the requirements which were needed to be fulfilled to be included in this survey, we hence assessed whether frameworks fulfilled four additional requirements:

- **Data streaming:** The framework should support both data streaming and batch processing. The difference between the two is that in data streaming records are processed in an element-wise fashion, allowing records to be send downstream a pipeline before the entire batch of records is processed (one example are [asynchronous functions](#)).
- **Multi-language support:** The framework should allow the implementation to be carried out in various programming languages to support a wide range of implementations.
- **Pipeline configuration as a file:** In order to generate and execute a pipeline based on the semantic description of that pipeline, it is desired that the pipeline configuration (i.e. layout and configuration values of transformation steps) can be formulated as a file. This allows for a translational mapping between the semantic pipeline description and the pipeline configuration file of the respective framework.
- **Branching pipeline:** Since strictly serial pipelines have a limited use, frameworks should allow branching of transformation steps, that is a step should allow zero or more subsequent steps.

Our assessment of the 10 frameworks according to these criteria is summarized in Table 3.

framework	data streaming	modular processors	multi-language support	pipeline configuration as file	branching pipeline
RDF-connect	✓	✓	✓	✓ rdf	✓
Linked Data Interactions Orchestrator	✓	✓	✗ Java	✓ yaml	✗
Apache Nifi	✓	✓	✗ Java	✓ xml, very verbose, can be shared and reused	✓
Semantic.works	✗ (batch-based, limited by SPARQL-queries to virtuoso)	✓	✗ Javascript	✓ json	● pipelines are currently linear, but is generally possible
Barnard59	✓	✓	● Javascript-based, other languages via wrappers	✓ rdf	?
LinkedPipes ETL		✓	✗ JVM	✓ JSON-LD	✓
Knime	● primarily batch-based, some streaming support	✓	✓	✗	✓

PART 2: SURVEY

1.4 DESIGN

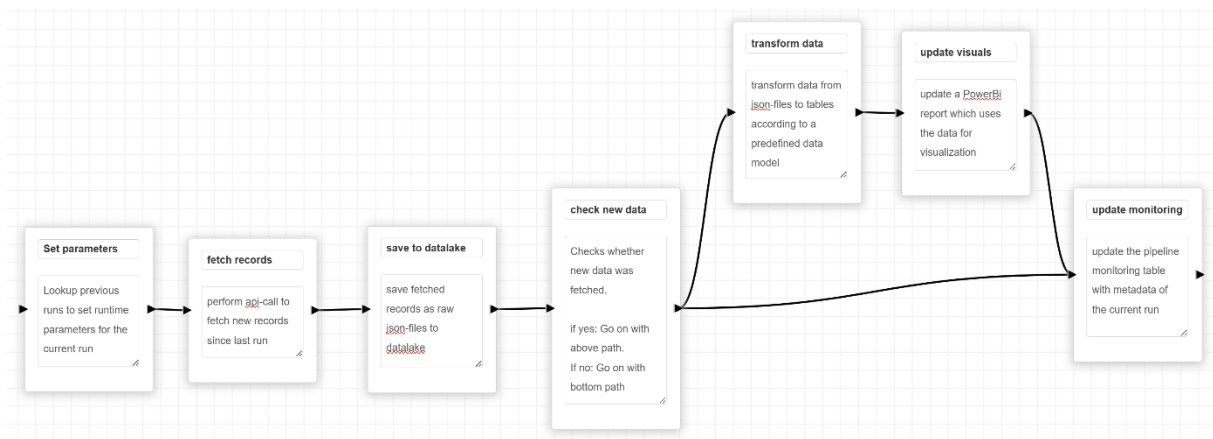
Based on our insights from Part 1, we conducted a web survey which empirically assesses which pipeline patterns and transformation steps are commonly applied by our survey participants. The survey was send to the following participants

Participants

- Sirius
- RedPencil
- ...

We used [drawflow](#) to let participants sketch out pipelines in from of a directed graph, whereby its nodes represent transformation steps and its edges represent data flows between those steps. The web survey was designed with [lab.js](#) and hosted on [open-lab](#), both are tooling for open and non-commercial research purposes. The web survey interface is shown in Figure 2.

Figure 2



1.5 RESULTS

Pipeline builder interface

Send to all partners of the user group

Took 15 minutes to complete

Hosted on openlab, free for scientific and noncommercial purposes

CONCLUSION